CIVIX 2.0 — Engineering Knowledge Base & Reference Manual

Document Status: Authoritative Technical Reference Manual
Audience: Software Engineers, Backend Developers, Database Administrators, System Operators
Authority Rule: Derived directly from `docs/00_CIVIX_CURRENT_STATE.md` and repository code inspection.

---

1. Key Invariants & Architectural Decision Records (ADRs)

- **INV-01 (Fact Integrity)**: Observed telemetry (CDRs, financial transactions, CCTV captures) is immutable and can never be altered by system inferences or investigator edits.
- **INV-08 (Supervisor Gatekeeper)**: AI predictions and investigator proposals cannot self-confirm into the Neo4j graph without explicit supervisor review.
- **INV-18 (Strict Predicates)**: All relationship predicates must match PostgreSQL `predicate_enum` definitions (`CALLED` , `MESSAGED` , `CO_LOCATED` , `TRANSFERRED_TO` , `REGISTERED_TO` , etc.). Free-text strings are rejected.
- **ADR-030 (Neo4j Graph Projection Rule)**: Neo4j is a read-side projection layer of PostgreSQL, synchronized asynchronously via transactional outbox events.
- **ADR-REM-01 (Assertion Schema)**: Proposal workflow uses existing `civix.assertion` table with `status` values `PROPOSED` , `ACCEPTED_BY_SUPERVISOR` , and `REJECTED` .
- **ADR-REM-02 (Unapproved Assertion Isolation)**: Assertions with status `PROPOSED` remain in PostgreSQL only and are **EXCLUDED** from Neo4j graph indexing.
- **ADR-REM-03 (Supervisor Edge Type)**: When an assertion is accepted by a supervisor, it is projected to Neo4j as an `:INVESTIGATOR_ASSERTED` edge carrying `assertion_id` , `created_by` , and `approved_by` .

---

2. PostgreSQL DDL Schema Definitions (`civix` Schema)

Below are the core DDL schema structures implemented in PostgreSQL (`scratch/civix_schema_utf8.sql`):

```
-- Core Entity Base Table
CREATE TABLE civix.entity (
    entity_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    entity_type VARCHAR(50) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Person Demographic Table
CREATE TABLE civix.person (
    entity_id UUID PRIMARY KEY REFERENCES civix.entity(entity_id) ON DELETE CASCADE,
    full_name VARCHAR(255) NOT NULL,
    gender VARCHAR(20),
    dob DATE,
    national_id VARCHAR(100),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Vehicle Registration Table
CREATE TABLE civix.vehicle (
    entity_id UUID PRIMARY KEY REFERENCES civix.entity(entity_id) ON DELETE CASCADE,
    plate_number VARCHAR(50) NOT NULL UNIQUE,
    make VARCHAR(100),
    model VARCHAR(100),
    color VARCHAR(50),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- GIS Location Table
CREATE TABLE civix.location (
    entity_id UUID PRIMARY KEY REFERENCES civix.entity(entity_id) ON DELETE CASCADE,
    location_name VARCHAR(255),
    geometry GEOMETRY(Point, 4326) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Spatiotemporal Event Table
CREATE TABLE civix.event (
    event_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    event_type VARCHAR(50) NOT NULL, -- CALL, TRANSACTION, CCTV_SIGHTING
    occurred_at TSTZRANGE NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Event Participant Table
CREATE TABLE civix.event_participant (
    event_participant_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    event_id UUID NOT NULL REFERENCES civix.event(event_id) ON DELETE CASCADE,
    entity_id UUID NOT NULL REFERENCES civix.entity(entity_id) ON DELETE CASCADE,
    participant_role VARCHAR(50) NOT NULL -- CALLER, CALLEE, SENDER, RECEIVER, CELL_TOWER
);

-- Investigator Assertions Table
CREATE TABLE civix.assertion (
    assertion_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    case_id UUID NOT NULL,
    subject_entity_id UUID NOT NULL REFERENCES civix.entity(entity_id),
    predicate VARCHAR(50) NOT NULL,
    object_entity_id UUID NOT NULL REFERENCES civix.entity(entity_id),
    status VARCHAR(50) NOT NULL DEFAULT 'PROPOSED', -- PROPOSED, ACCEPTED_BY_SUPERVISOR, REJECTED
    asserted_by VARCHAR(50) NOT NULL DEFAULT 'INVESTIGATOR',
    justification TEXT,
    created_by UUID NOT NULL,
    approved_by UUID,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

```
-- Transactional Outbox Table for Neo4j CDC
CREATE TABLE civix.outbox (
    outbox_id BIGSERIAL PRIMARY KEY,
    event_type VARCHAR(100) NOT NULL,
    payload JSONB NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    processed_at TIMESTAMPTZ
);
```

---

3. Complete API Directory (`civix_api/routers/`)

Router Module	Path Prefix	Key Endpoints & Parameters	Purpose & Output	Authorization Scope

`cases.py`	`/api/v1/cases`	`GET /`
		`POST /`
		`GET /{id}`
		`GET /{id}/graph?depth=3&limit=500`
			Lists cases, creates new case, fetches case details, and retrieves multi-hop Cypher graph nodes/edges payload.	Case ACL (`READ` , `WRITE` , `ADMIN`)

`assertions.py`	`/api/v1/cases`	`POST /{id}/assertions`
		`POST /{id}/assertions/{aid}/review`
		`GET /{id}/assertions/proposed`
			Submits relationship proposal (`PROPOSED`), reviews proposal (`ACCEPTED_BY_SUPERVISOR` / `REJECTED`), and lists pending proposals.	`WRITE` to propose, `ADMIN` to review

`spatial.py`	`/api/v1/spatial`	`GET /events?bbox=...`
		`GET /centroids`
		`GET /movement?entity_id=...`
			Returns spatiotemporal GIS events, cell tower centroids, and vehicle trajectory vectors.	Case `READ`

`cctv.py`	`/api/v1/cctv`	`GET /streams`
		`POST /process`
		`GET /sightings?plate=...`
			Lists 25 CCTV camera streams, triggers YOLOv8+SORT ANPR processing, and queries vehicle sightings.	Case `READ` / `WRITE`

`telecom.py`	`/api/v1/telecom`	`GET /matrix?case_id=...`
		`GET /colocation`
		`GET /analysis`
			Generates inter-entity call volume matrix, cell tower co-location reports, and night call analysis.	Case `READ`

`evidence.py`	`/api/v1/evidence`	`POST /upload`
		`GET /vault`
		`GET /{id}/entities`
			Uploads PDF evidence documents, lists evidence vault files, and returns Gemini Flash 1.5 extracted entity tags.	Case `READ` / `WRITE`

`leads.py`	`/api/v1/cases`	`GET /{id}/leads` `POST /{id}/leads/score`	Triggers 70-feature SQL CTE extractor (`feature_extractor.py`), runs XGBoost model, and returns candidate risk scores.	Case `READ` / XGBoost ML Model
`biometric.py`	`/api/v1/biometric`	`POST /match` `GET /profiles`	Executes facial/fingerprint embedding comparison and returns similarity scores.	Prototype / Demo Endpoint
`search.py`	`/api/v1/search`	`GET /query?q=...` `POST /nlp`	Natural language Ask CIVIX search converting text questions into SQL/Cypher queries.	Case `READ`
`users.py`	`/api/v1/users`	`POST /login` `GET /me`	Authenticates investigator credentials, returns JWT bearer token, and retrieves user profile details.	Public / Token Auth

4. Data Pipeline & Outbox CDC Worker Internals

```
PostgreSQL Transaction (`civix.assertion`)
        |
        v Status updated to 'ACCEPTED_BY_SUPERVISOR'
Trigger Inserts Outbox Record into `civix.outbox`
(event_type = 'ASSERTION_ACCEPTED', payload = {assertion_id, subject_id, predicate, object_id, case_id})
        |
        v
`worker/outbox_worker.py` (Async Python Polling Loop)
        | Reads unprocessed records where `processed_at IS NULL`
        v
`services/neo4j_projection.py` --> Executes Cypher MERGE:
  MATCH (c:Case {case_id: $case_id})
  MERGE (s:Entity {entity_id: $subject_id})
  MERGE (o:Entity {entity_id: $object_id})
  MERGE (s)-[r:INVESTIGATOR_ASSERTED {
      assertion_id: $assertion_id,
      predicate: $predicate,
      created_by: $created_by,
      approved_by: $approved_by,
      timestamp: $timestamp
  }]->(o)
        |
        v
PostgreSQL Updates Outbox Record (`processed_at = NOW()`)
```

5. Developer Troubleshooting & Diagnostic Guide

Issue 1: Neo4j Graph Not Updating After Assertion Creation

- **Root Cause**: The assertion status is set to `PROPOSED` (which is isolated from Neo4j per ADR-REM-02), or the Python outbox worker is not running.
- **Diagnostic Steps**:
- Check assertion status in PostgreSQL: `SELECT status FROM civix.assertion WHERE assertion_id = '...';`
- Verify outbox processing status: `SELECT processed_at FROM civix.outbox WHERE event_type = 'ASSERTION_ACCEPTED';`
- Ensure `worker/outbox_worker.py` service process is active.

Issue 2: XGBoost Candidate Feature Extractor Returns Empty Dict

- **Root Cause**: The candidate entity UUID has no caller/sender records in `civix.event_participant`.
- **Diagnostic Steps**:
- Inspect `feature_extractor.py` CTE queries.
- Verify participant roles: candidate MUST have role `CALLER` or `SENDER` in `civix.event_participant`.

Issue 3: PostGIS Geometry Queries Return Null

- **Root Cause**: Missing spatial SRID assignment or invalid coordinate ordering.
- **Diagnostic Steps**:
- Ensure point geometry uses SRID 4326: `ST_SetSRID(ST_MakePoint(longitude, latitude), 4326)`.
- Remember PostGIS takes `(longitude, latitude)` order, NOT `(latitude, longitude)`.